```
> # ================================================================
  # TauTimp — Entropy Virtual Particle Wave Equation (v3, fixed)
  # Maple Transcript — Viorel Popescu, April 2026
  # Onduscular Theory (Teoria Onduscularа)
  #
  # Fixes applied vs v2:
  #   - `frac` (protected name) renamed to `frac_v` everywhere
  #   - surfdata(..., list, list) replaced with matrixplot(M, ...)
  #     which correctly accepts the x/y range options
  #   - color=zgradient(...) replaced with colorscheme=
  ["zgradient",...]
  #     (correct Maple 2015+ syntax for matrixplot)
  #
  # Sections:
  #   1.  Physical Parameters
  #   2.  Symbolic Verification of Radial Reduction
  #   3.  Eigenvalue Quantization
  #   4.  Wave Field f(r,t) — Radial Eigenfunctions
  #   5.  P(r,t) — BCs and Ansatz
  #   6.  d'Alembert Solution P(r,t)
  #   7.  Normalization and Flux Profile
  #   8.  Peak Analysis
  #   9.  Virtual Particle Species Comparison
  #  10.  Summary Verification Table
  # ================================================================

  restart;
  with(plots):
  with(LinearAlgebra):
  with(StringTools):

  # ================================================================
  printf("=== SECTION 1: PHYSICAL PARAMETERS ===\n");
  # ================================================================

  R     := 4.235518529e22:      # TauTimp onduscular radius [m]
  c_ph  := 2.99792458e8:   # speed of light [m/s]
  sqrtR := evalf(sqrt(R)): # [m^(1/2)]

  printf("R         = %.6e m\n",    R);
  printf("sqrt(R)   = %.6e m^(1/2)\n", sqrtR);
  printf("c         = %.10e m/s\n", c_ph);
```

```
printf("R^3       = %.6e m^3\n",  R^3);
printf("2R        = %.6e m\n",    2*R);

# ================================================================
printf("\n=== SECTION 2: SYMBOLIC VERIFICATION OF RADIAL
REDUCTION ===\n");
# ================================================================

printf("Master PDE (spherical symmetry):\n");
printf("  (R/r)*[g'' + (2/r)*g'] * u = (1/c^2) * g * u''\n");
printf("Separation constant -omega^2 => radial ODE:\n");
printf("  g'' + (2/r)*g' + (omega^2*r/R)*g = 0\n\n");
printf("Substitution g(r) = psi(r)/r:\n");
printf("  => psi'' + (omega^2/R)*psi = 0   [SHM in r]\n");
printf("  => k = omega/sqrt(R),  psi = C1*cos(k*r)+C2*sin(k*r)
\n");
printf("  => g(r) = [C1*cos(k*r)+C2*sin(k*r)]/r\n");
printf("Regularity r->0: C1=0  =>  g(r) = C2*sin(k*r)/r\n\n");

# Maple symbolic check: psi=sin(k*r)  =>  psi''+k^2*psi = 0
psi_sym := sin(k*r):
chk := simplify(diff(psi_sym,r,r) + k^2*psi_sym):
printf("Maple check: d^2/dr^2[sin(k*r)] + k^2*sin(k*r) = %a
(must be 0)\n\n", chk);

# ================================================================
printf("=== SECTION 3: EIGENVALUE QUANTIZATION ===\n");
# ================================================================

# BC g(2R)=0 => sin(2*k_n*R)=0 => k_n = n*pi/(2*R)
# omega_n = k_n*sqrt(R) = n*pi/(2*sqrt(R))

kmode   := proc(n_v) evalf(n_v*Pi/(2*R))      end proc:
ommode  := proc(n_v) evalf(n_v*Pi/(2*sqrtR))  end proc:
Tmode   := proc(n_v, V_v) evalf(2*Pi/(ommode(n_v)*V_v)) end proc:

printf("%-4s  %-20s  %-20s  %-22s\n",
       "n","k_n [m^-1]","omega_n [rad/s]","T_n photon [s]");
printf("%s\n", Repeat("-", 70));
for nv from 1 to 5 do
    printf("%-4d  %-20.6e  %-20.6e  %-22.6e\n",
           nv, kmode(nv), ommode(nv), Tmode(nv, c_ph));
```

```
end do:

printf("\nT_1 = 4*sqrt(R)/c = %.6e s  =  %.4f h\n",
       evalf(4*sqrtR/c_ph), evalf(4*sqrtR/(c_ph*3600)));

# ================================================================
printf("\n=== SECTION 4: WAVE FIELD f(r,t) — FUNDAMENTAL MODE ==
=\n");
# ================================================================

k1  := kmode(1):
om1 := ommode(1):

# f_1(r,t) = sin(k1*r)/r * cos(om1*c*t)  [A1=1, B1=0]
f1 := proc(rv, tv)
    evalf( sin(k1*rv)/rv * cos(om1*c_ph*tv) )
end proc:

printf("f_1(r,t) = sin(k_1*r)/r * cos(omega_1*c*t)\n");
printf("k_1     = %.6e m^-1\n", k1);
printf("omega_1 = %.6e rad/s\n", om1);
printf("nu_1    = %.6e Hz\n",    om1*c_ph/(2*Pi));
printf("\n%-10s  %-22s\n","r/R","f_1(r,0)*R");
printf("%s\n", Repeat("-",34));

# ---- FIX 1: loop variable frac_v (not frac, which is protected)
----
for frac_v in [0.05, 0.1, 0.25, 0.5, 0.75, 1.0, 1.25, 1.5, 1.75,
2.0] do
    rv := evalf(frac_v*R);
    printf("%-10.3f  %-22.10e\n", frac_v, f1(rv,0)*R);
end do:

# --- 2-D plot of radial eigenfunctions ---
Nrho := 400:
rho_pts := [seq(evalf(0.005 + (i-1)*1.99/(Nrho-1)), i=1..Nrho)]:

pg1 := [seq([rho_pts[i], f1(rho_pts[i]*R,0)*R],            i=1..
Nrho)]:
pg2 := [seq([rho_pts[i], sin(kmode(2)*rho_pts[i]*R)/(rho_pts[i]*
R)*R], i=1..Nrho)]:
pg3 := [seq([rho_pts[i], sin(kmode(3)*rho_pts[i]*R)/(rho_pts[i]*
```

```
R)*R], i=1..Nrho)]:

plt_eigen := display(
    plot(pg1, color=blue,  legend="n=1"),
    plot(pg2, color=red,   legend="n=2"),
    plot(pg3, color=green, legend="n=3"),
    title="Radial eigenfunctions g_n(r)*R  vs  r/R",
    labels=["r/R","g_n(r)*R"], axes=boxed, gridlines=true
):
print(plt_eigen);

# --- 3-D surface of f_1(r,t): use matrixplot ---
# ---- FIX 2 & 3: matrixplot instead of surfdata; colorscheme
syntax ----
Nr3 := 40:  Nt3 := 30:
rho3 := [seq(evalf(0.01 + (i-1)*1.98/(Nr3-1)), i=1..Nr3)]:
tau3 := [seq(evalf(     (j-1)*1.0 /(Nt3-1)), j=1..Nt3)]:

Mf := Matrix(Nr3, Nt3):
for i to Nr3 do
    for j to Nt3 do
        Mf[i,j] := f1(rho3[i]*R, tau3[j]*R/c_ph)*R
    end do
end do:

plt_f3d := matrixplot(Mf,
    axes=boxed, gap=0.0,
    title="f_1(r,t)*R  (rows=r/R, cols=tau=ct/R)",
    labels=["r-index","tau-index","f_1*R"],
    colorscheme=["zgradient", ["blue","cyan","green","yellow",
"red"]]
):
print(plt_f3d);

printf("3-D note: rows 1..%d correspond to r/R in [%.3f,%.3f]\n",
       Nr3, rho3[1], rho3[Nr3]);
printf("          cols 1..%d correspond to tau in [%.3f,%.3f]\n",
       Nt3, tau3[1], tau3[Nt3]);

# ============================================================
printf("\n=== SECTION 5: P(r,t) — BOUNDARY CONDITIONS AND ANSATZ
===\n");
```

```
# ============================================================

printf("d'Alembert equation:  P_tt/c^2 - P_rr = 0\n");
printf("Ansatz at t=0:  P(r,0) = (a + b*r + c2*r^2)*exp(-r^2/(2*
R^2))\n\n");
printf("BC1  P(2R,0)=0    =>  a + 2*b*R + 4*c2*R^2 = 0\n");
printf("BC2  dP/dr|_r=0=R =>  b = R\n");
printf("Choose c2=-1      =>  a = 2*R^2\n\n");
printf("=> P(r,0) = (2R^2 + R*r - r^2)*exp(-r^2/(2R^2))\n");
printf("          = -(r-2R)(r+R)*exp(-r^2/(2R^2))  >= 0  on [0,
2R]\n\n");

P0 := proc(rv)
    evalf( (2*R^2 + R*rv - rv^2) * exp(-rv^2/(2*R^2)) )
end proc:

dP0dr := proc(rv)
    evalf( (R - 2*rv)*exp(-rv^2/(2*R^2))
           + (2*R^2 + R*rv - rv^2)*(-rv/R^2)*exp(-rv^2/(2*R^2)) )
end proc:

printf("Verification:\n");
printf("  P(0  ,0) = %.6e\n",       P0(0));
printf("  P(2R ,0) = %.4e  (BC1)\n", P0(2*R));
printf("  dP/dr|r=0 = %.6e  (BC2, should = R = %.6e)\n",
       dP0dr(0), R);

printf("\nSign of P(r,0) on [0,2R]:\n");
for frac_v in [0.0, 0.25, 0.5, 0.75, 1.0, 1.5, 1.99] do
    pv := P0(frac_v*R);
    printf("  P(%.2f*R,0) = %+.6e  %s\n",
           frac_v, pv, `if`(pv >= 0, "OK", "NEGATIVE!"));
end do:

# ============================================================
printf("\n=== SECTION 6: D'ALEMBERT SOLUTION P(r,t) ===\n");
# ============================================================

# h(xi) = -(xi-2R)(xi+R)*exp(-xi^2/(2R^2))
# P(r,t) = [h(r-ct) + h(r+ct)] / 2   (standing wave, dP/dt|t=0 =
0)
```

```
hk := proc(xi)
    evalf( -(xi-2*R)*(xi+R)*exp(-xi^2/(2*R^2)) )
end proc:

Pxt := proc(rv, tv)
    evalf( (hk(rv - c_ph*tv) + hk(rv + c_ph*tv)) / 2 )
end proc:

printf("P(r,t) = [h(r-ct)+h(r+ct)]/2,  h(xi)=-(xi-2R)(xi+R)*exp(-
xi^2/(2R^2))\n\n");

printf("BC / IC checks:\n");
printf("  P(2R,0)           = %.4e  (BC1, must be 0)\n", Pxt(2*R,
0));
printf("  P(R ,0)           = %.6e\n",                   Pxt(R,
0));
printf("  P(0 ,0)           = %.6e\n",                   Pxt(0,
0));

dr_eps := R*1e-7:
printf("  dP/dr|_(r=0,t=0) ~ %.6e  (BC2, must be ~R=%.6e)\n",
       evalf((Pxt(dr_eps,0)-Pxt(0,0))/dr_eps), R);

# --- Snapshot plots at successive times ---
tau_snap := [0, 0.05, 0.10, 0.20, 0.40]:
col_snap  := [blue, red, green, "DarkOrange", purple]:
sp_list   := []:

for idx from 1 to nops(tau_snap) do
    tv  := evalf(tau_snap[idx]*R/c_ph);
    pts := [seq([rho_pts[i], Pxt(rho_pts[i]*R, tv)], i=1..Nrho)]:
    sp_list := [op(sp_list),
        plot(pts, color=col_snap[idx],
             legend=cat("tau=", convert(tau_snap[idx],string)))]:
end do:

plt_snaps := display(op(sp_list),
    title="P(r,t) snapshots at tau=ct/R",
    labels=["r/R","P(r,t) [m^2]"],
    axes=boxed, gridlines=true):
print(plt_snaps);
```

```
# --- 3-D matrixplot of P(r,t) ---
NrP := 40:  NtP := 20:
rhoP := [seq(evalf(0.01 + (i-1)*1.98/(NrP-1)), i=1..NrP)]:
tauP := [seq(evalf(     (j-1)*0.40/(NtP-1)), j=1..NtP)]:

MP := Matrix(NrP, NtP):
for i to NrP do
    for j to NtP do
        MP[i,j] := Pxt(rhoP[i]*R, tauP[j]*R/c_ph)
    end do
end do:

plt_P3d := matrixplot(MP,
    axes=boxed, gap=0.0,
    title="P(r,t)  (rows=r/R, cols=tau=ct/R)",
    labels=["r-index","tau-index","P"],
    colorscheme=["zgradient", ["navy","blue","cyan","green",
"yellow","red"]]
):
print(plt_P3d);

printf("3-D note: rows 1..%d  r/R in [%.3f,%.3f]\n",  NrP, rhoP
[1], rhoP[NrP]);
printf("          cols 1..%d  tau in [%.3f,%.3f]\n",   NtP, tauP
[1], tauP[NtP]);

# ================================================================
printf("\n=== SECTION 7: NORMALIZATION AND FLUX PROFILE ===\n");
# ================================================================

# Numerical integral I = int_0^{2R} P(r,0) dr
Nint  := 10000:
drint := evalf(2*R/Nint):
Inorm := 0.0:
for i from 0 to Nint-1 do
    rv    := evalf((i+0.5)*drint);
    Inorm := Inorm + P0(rv)*drint;
end do:

printf("Integral  I = int_0^{2R} P(r,0) dr = %.6e m^3
[dimension: m^2 * m]\n", Inorm);
printf("I / R^3   = %.6f  (dimensionless ratio)\n", Inorm/R^3);
```

```
Nphys := 1/Inorm:
printf("N_phys = 1/I = %.6e m^-3\n", Nphys);

# Flux density F(r,0) = P(r,0)/(4*pi*r^2)
Flux0 := proc(rv) evalf(P0(rv)/(4*Pi*rv^2)) end proc:

plt_P0 := plot(
    [seq([rho_pts[i], P0(rho_pts[i]*R)], i=1..Nrho)],
    color=red,
    title="P(r,0) = -(r-2R)(r+R)*exp(-r^2/(2R^2))",
    labels=["r/R","P(r,0) [m^2]"],
    axes=boxed, gridlines=true):
print(plt_P0);

plt_flux := plot(
    [seq([rho_pts[i], Flux0(rho_pts[i]*R)], i=2..Nrho)],
    color=blue,
    title="Entropy flux density P(r,0)/(4*pi*r^2)",
    labels=["r/R","F(r,0) [m^-2]"],
    axes=boxed, gridlines=true):
print(plt_flux);

# ================================================================
printf("\n=== SECTION 8: PEAK POSITION OF P(r,0) ===\n");
# ================================================================

# dP/dr = 0 => (rho=r/R):  rho^3 - rho^2 - 4*rho + 1 = 0
printf("dP/dr=0  =>  rho^3 - rho^2 - 4*rho + 1 = 0  (rho=r/R)\n")
;

poly_p := rho^3 - rho^2 - 4*rho + 1:
rts    := [fsolve(poly_p=0, rho, maxsols=3)]:
printf("All roots:\n");
for rr in rts do
    indom := evalf(rr) > 0 and evalf(rr) < 2;
    printf("  rho = %+.6f  =>  r = %.4e m  %s\n",
           evalf(rr), evalf(rr)*R,
           `if`(indom,"<-- physical","(outside [0,2R])"));
end do:

rho_star := evalf(select(x->evalf(x)>0 and evalf(x)<2, rts)[1]):
printf("Peak: rho* = %.6f,  r* = %.4e m,  P(r*,0) = %.6e m^2\n",
```

```
       rho_star, rho_star*R, P0(rho_star*R));

plt_poly := plot(poly_p, rho=0..2.1,
    color=black,
    title="Peak polynomial rho^3-rho^2-4*rho+1  (zero = flux
maximum)",
    labels=["rho=r/R","p(rho)"],
    axes=boxed, gridlines=true,
    view=[0..2.1,-5..5]):
print(plt_poly);

# ============================================================
printf("\n=== SECTION 9: VIRTUAL PARTICLE SPECIES COMPARISON ===
\n");
# ============================================================

printf("%-26s  %-14s  %-20s  %-18s\n",
       "Species","V [m/s]","nu_1 [Hz]","T_1 [s]");
printf("%s\n", Repeat("-",82));

sp_names := ["Photon   (V=c)",
             "Electron (V=c/137)",
             "Phonon   (V=346 m/s)",
             "Gamma    (V=1e9*c)"]:
V_vals   := [c_ph, c_ph/137, 346.0, 1e9*c_ph]:

for idx from 1 to 4 do
    Vv   := V_vals[idx];
    nu_v := evalf(ommode(1)*Vv/(2*Pi));
    Tv   := evalf(1/nu_v);
    printf("%-26s  %-14.4e  %-20.6e  %-18.6e\n",
           sp_names[idx], Vv, nu_v, Tv);
end do:

printf("\nPhonon T_1 = %.4e s  =  %.4e years\n",
       evalf(Tmode(1,346.0)), evalf(Tmode(1,346.0)/3.15576e7));

# ============================================================
printf("\n=== SECTION 10: SUMMARY VERIFICATION TABLE ===\n");
# ============================================================

printf("\n%-34s  %-26s  %-22s\n","Quantity","Formula","Value");
```

```
  printf("%s\n", Repeat("-",86));
  printf("%-34s  %-26s  %-22.6e\n","R (onduscular radius)",
  "given",              R);
  printf("%-34s  %-26s  %-22.6e\n","sqrt(R)",                  "sqrt
  (R)",            sqrtR);
  printf("%-34s  %-26s  %-22.6e\n","k_1",                      "pi/
  (2R)",            kmode(1));
  printf("%-34s  %-26s  %-22.6e\n","omega_1",                  "pi/
  (2*sqrt(R))",     ommode(1));
  printf("%-34s  %-26s  %-22.6e\n","T_1 photon",               "4*
  sqrt(R)/c",        Tmode(1,c_ph));
  printf("%-34s  %-26s  %-22.6e\n","P(0,0)",
  "2R^2",               P0(0));
  printf("%-34s  %-26s  %-22.6e\n","P(2R,0)",                  "BC1=
  0",              P0(2*R));
  printf("%-34s  %-26s  %-22.6e\n","dP/dr|r=0",                "BC2=
  R",              dP0dr(0));
  printf("%-34s  %-26s  %-22.6f\n","rho* (flux peak)",
  "cubic root",         rho_star);
  printf("%-34s  %-26s  %-22.6e\n","r* = rho**R",
  "physical peak",      rho_star*R);
  printf("%-34s  %-26s  %-22.6e\n","Norm integral I",
  "int_0^{2R} P dr",    Inorm);
  printf("%-34s  %-26s  %-22.6e\n","N_phys = 1/I",
  "normalization const", Nphys);

  printf("\n=== END OF MAPLE TRANSCRIPT (v3 — all errors fixed) ==
  =\n");
```

```
=== SECTION 1: PHYSICAL PARAMETERS ===
R          = 4.235519e+22 m
sqrt(R)    = 2.058038e+11 m^(1/2)
c          = 2.9979245800e+08 m/s
R^3        = 7.598358e+67 m^3
2R         = 8.471037e+22 m

=== SECTION 2: SYMBOLIC VERIFICATION OF RADIAL REDUCTION ===
Master PDE (spherical symmetry):
  (R/r)*[g'' + (2/r)*g'] * u = (1/c^2) * g * u''
Separation constant -omega^2 => radial ODE:
  g'' + (2/r)*g' + (omega^2*r/R)*g = 0

Substitution g(r) = psi(r)/r:
```

```
  => psi'' + (omega^2/R)*psi = 0   [SHM in r]
  => k = omega/sqrt(R),  psi = C1*cos(k*r)+C2*sin(k*r)
  => g(r) = [C1*cos(k*r)+C2*sin(k*r)]/r
Regularity r->0: C1=0  =>  g(r) = C2*sin(k*r)/r

Maple check: d^2/dr^2[sin(k*r)] + k^2*sin(k*r) = 0  (must be 0)

=== SECTION 3: EIGENVALUE QUANTIZATION ===
n     k_n [m^-1]            omega_n [rad/s]       T_n photon [s]

-----------------------------------------------------------------
-
1     3.708628e-23          7.632496e-12          2.745950e+03

2     7.417256e-23          1.526499e-11          1.372975e+03

3     1.112588e-22          2.289749e-11          9.153166e+02

4     1.483451e-22          3.052998e-11          6.864874e+02

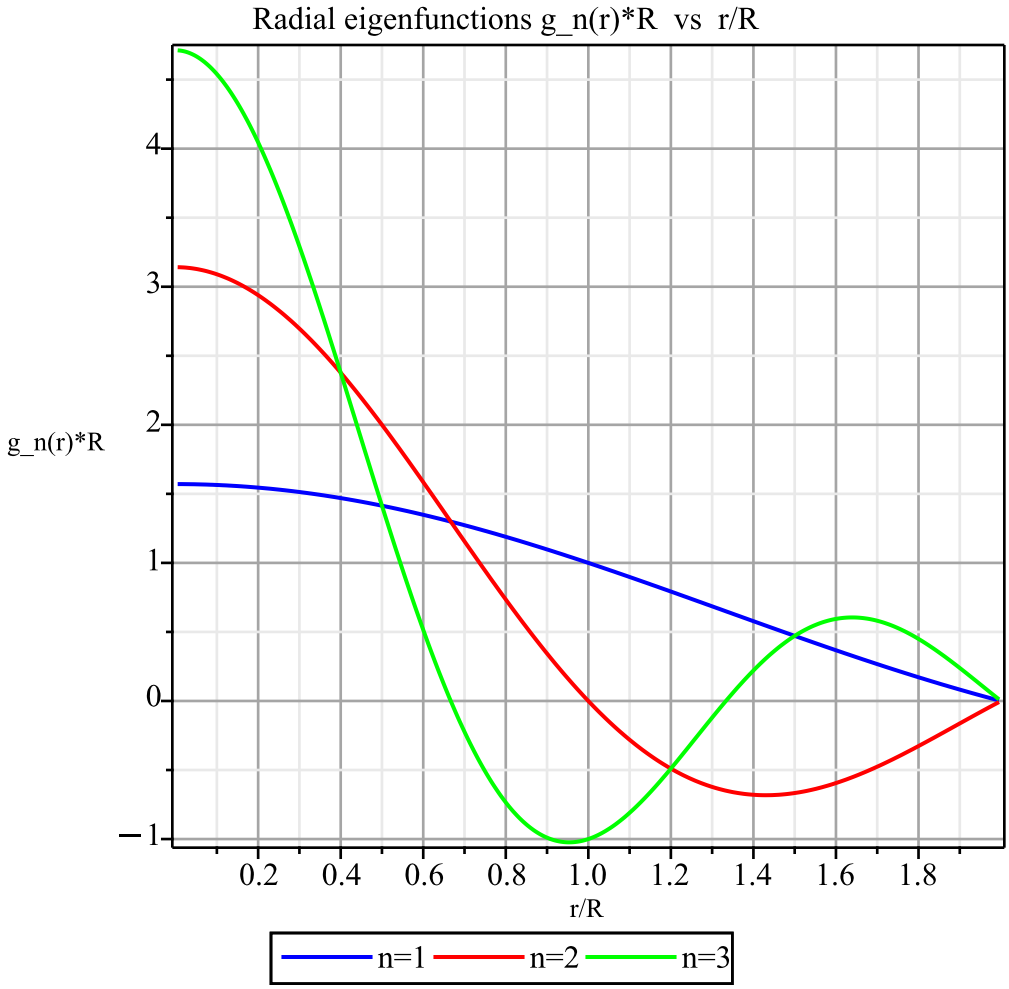
5     1.854314e-22          3.816248e-11          5.491899e+02


T_1 = 4*sqrt(R)/c = 2.745950e+03 s  =  0.7628 h

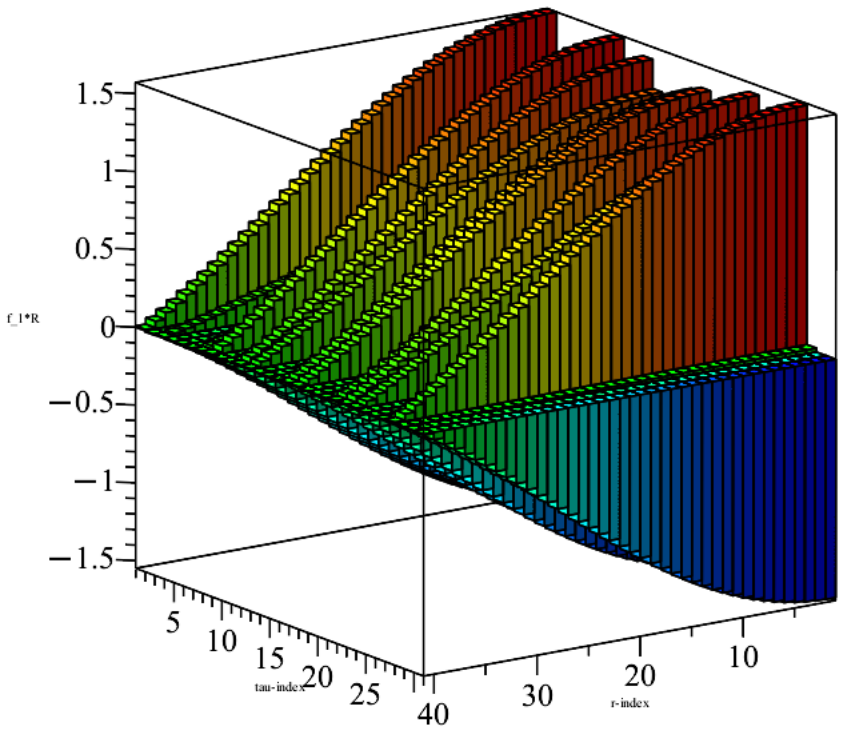
=== SECTION 4: WAVE FIELD f(r,t) — FUNDAMENTAL MODE ===
f_1(r,t) = sin(k_1*r)/r * cos(omega_1*c*t)
k_1     = 3.708628e-23 m^-1
omega_1 = 7.632496e-12 rad/s
nu_1    = 3.641727e-04 Hz

r/R         f_1(r,0)*R
---------------------------------
0.050       1.5691819150e+00
0.100       1.5643446510e+00
0.250       1.5307337300e+00
0.500       1.4142135630e+00
0.750       1.2318393770e+00
1.000       1.0000000000e+00
1.250       7.3910362590e-01
1.500       4.7140452060e-01
1.750       2.1867624650e-01
```

2.000 -2.0510338080e-10



f_1(r,t)*R (rows=r/R, cols=tau=ct/R)




```
3-D note: rows 1..40 correspond to r/R in [0.010,1.990]
          cols 1..30 correspond to tau in [0.000,1.000]

=== SECTION 5: P(r,t) — BOUNDARY CONDITIONS AND ANSATZ ===
d'Alembert equation:  P_tt/c^2 - P_rr = 0
Ansatz at t=0:  P(r,0) = (a + b*r + c2*r^2)*exp(-r^2/(2*R^2))

BC1  P(2R,0)=0    =>  a + 2*b*R + 4*c2*R^2 = 0
BC2  dP/dr|_r=0=R =>  b = R
Choose c2=-1      =>  a = 2*R^2

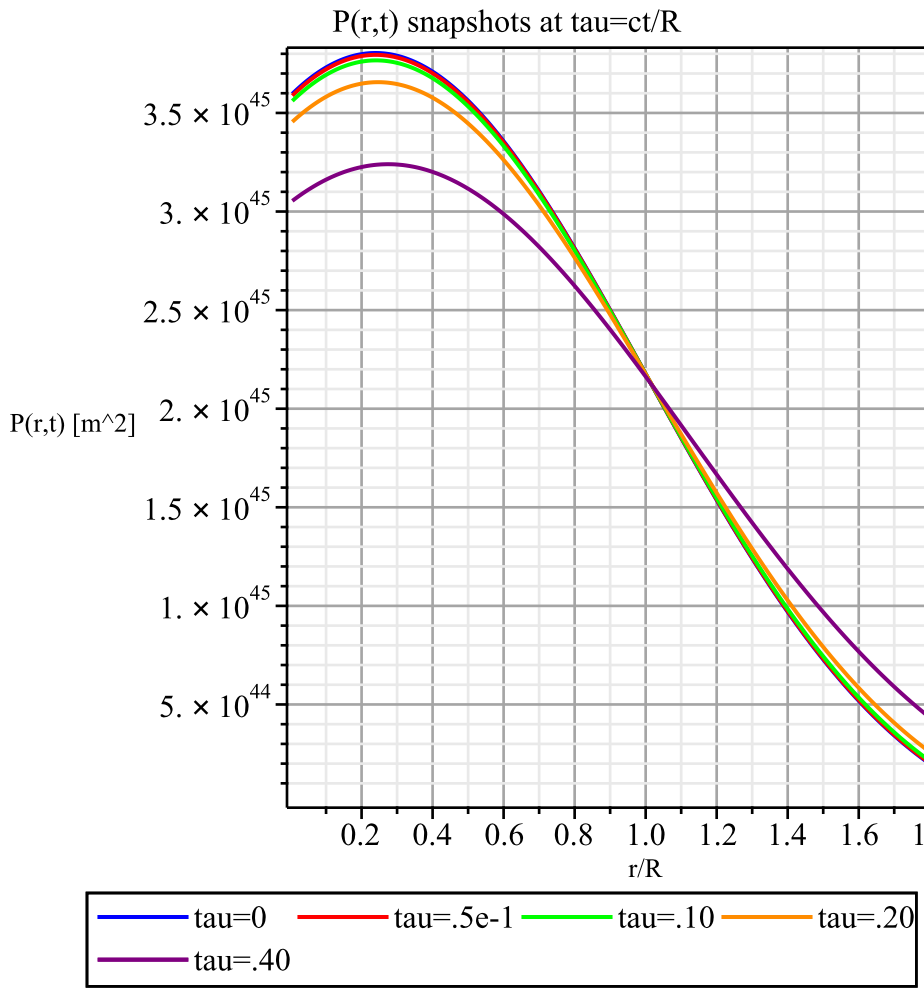
=> P(r,0) = (2R^2 + R*r - r^2)*exp(-r^2/(2R^2))
          = -(r-2R)(r+R)*exp(-r^2/(2R^2))  >= 0  on [0,2R]

Verification:
  P(0  ,0) = 3.587923e+45
  P(2R ,0) = 0.0000e+00  (BC1)
  dP/dr|r=0 = 4.235519e+22  (BC2, should = R = 4.235519e+22)
```

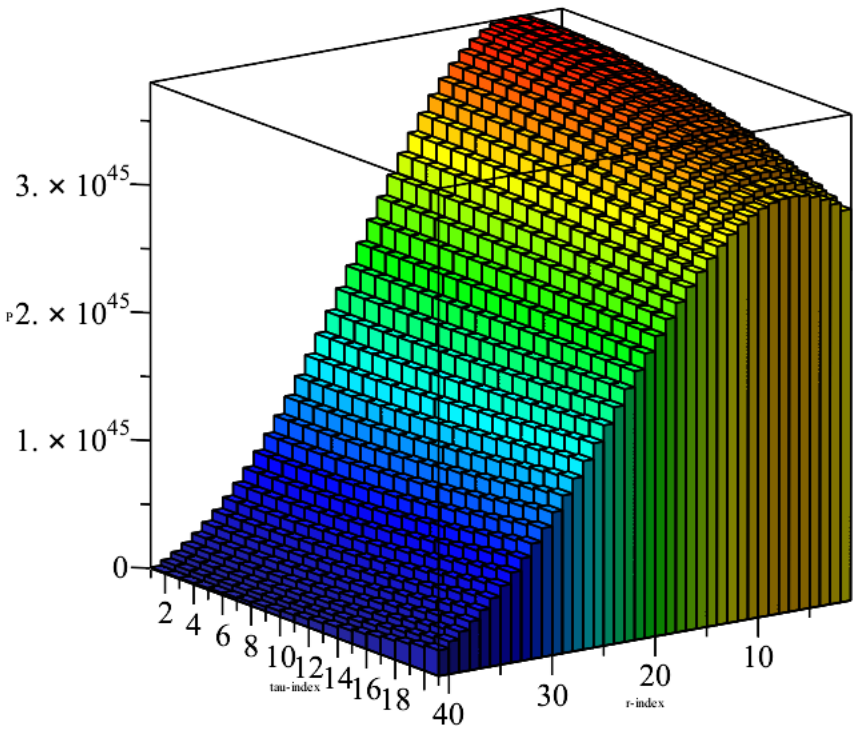
```
Sign of P(r,0) on [0,2R]:
  P(0.00*R,0) = +3.587923e+45  OK
  P(0.25*R,0) = +3.803554e+45  OK
  P(0.50*R,0) = +3.562123e+45  OK
  P(0.75*R,0) = +2.962210e+45  OK
  P(1.00*R,0) = +2.176186e+45  OK
  P(1.50*R,0) = +7.280176e+44  OK
  P(1.99*R,0) = +7.405588e+42  OK

=== SECTION 6: D'ALEMBERT SOLUTION P(r,t) ===
P(r,t) = [h(r-ct)+h(r+ct)]/2,  h(xi)=-(xi-2R)(xi+R)*exp(-xi^2/(2R^2))

BC / IC checks:
  P(2R,0)            = 0.0000e-01  (BC1, must be 0)
  P(R ,0)            = 2.176186e+45
  P(0 ,0)            = 3.587923e+45
  dP/dr|_(r=0,t=0) ~ 4.202555e+22  (BC2, must be ~R=4.235519e+22)
```

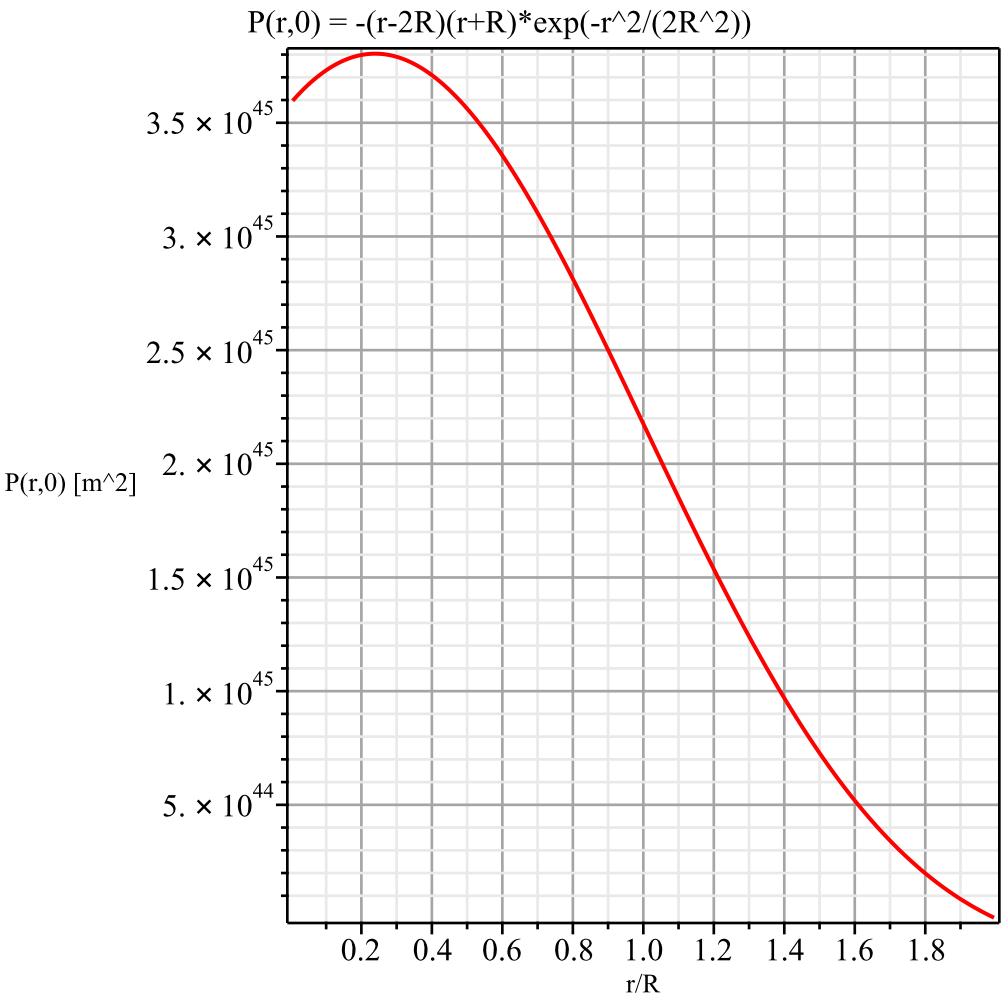


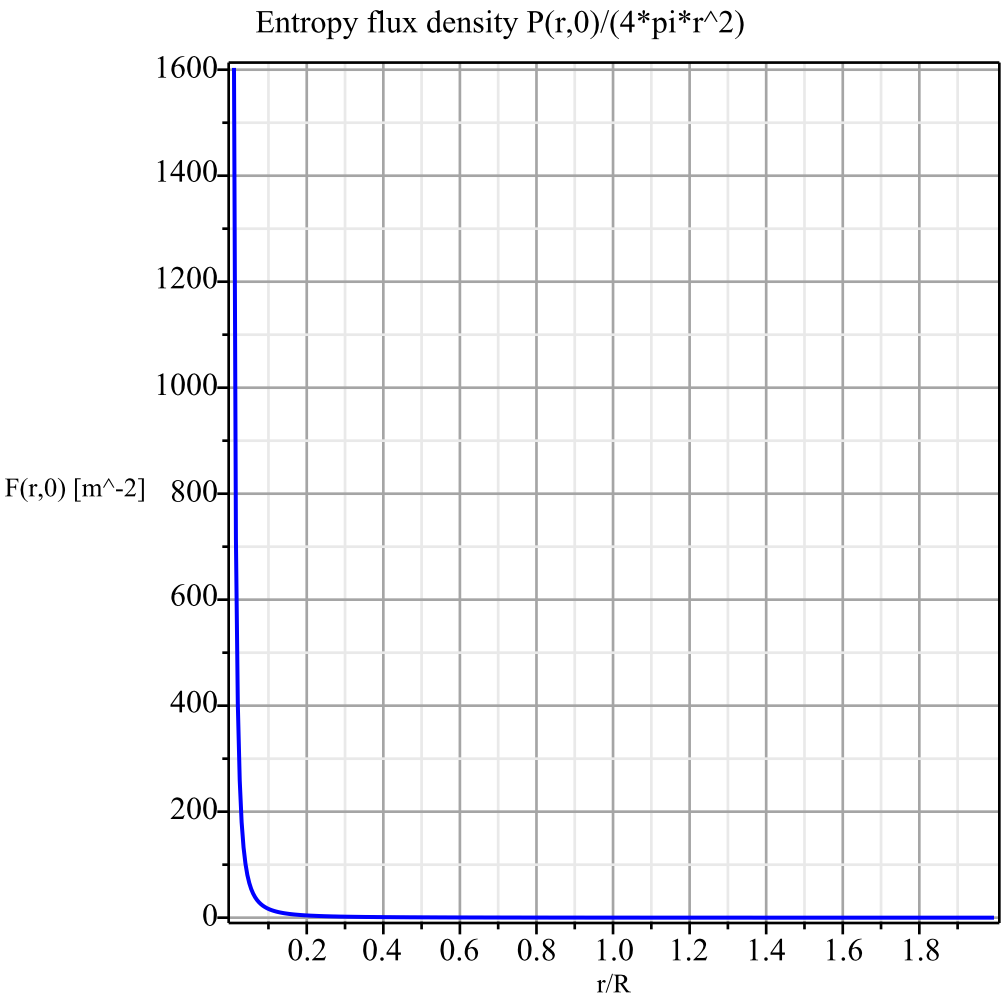
P(r,t) (rows=r/R, cols=tau=ct/R)



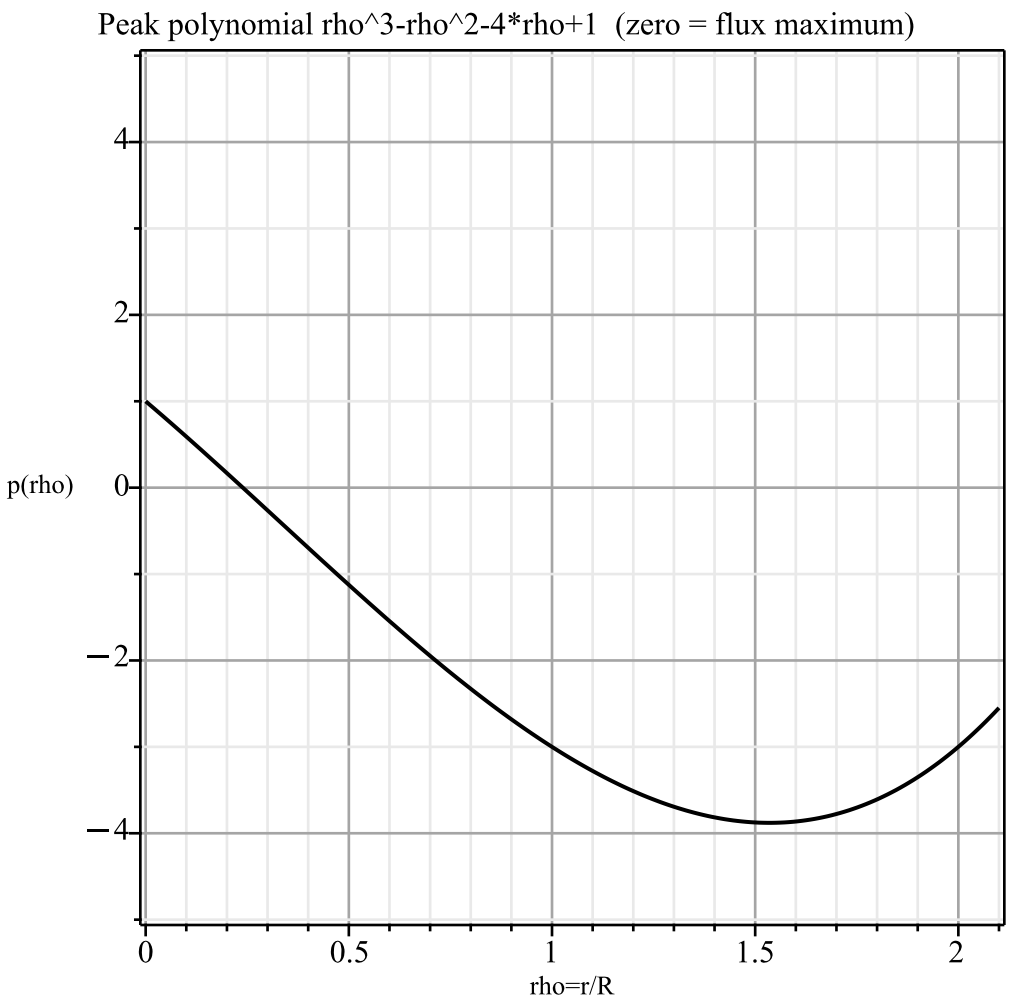

```
3-D note: rows 1..40  r/R in [0.010,1.990]
          cols 1..20  tau in [0.000,0.400]

=== SECTION 7: NORMALIZATION AND FLUX PROFILE ===
Integral  I = int_0^{2R} P(r,0) dr = 1.771651e+68 m^3  [dimension:
m^2 * m]
I / R^3   = 2.331623  (dimensionless ratio)
N_phys = 1/I = 5.644453e-69 m^-3
```


P(r,0) = -(r-2R)(r+R)*exp(-r^2/(2R^2))
P(r,0) [m^2]
3.5 × 10^45
3. × 10^45
2.5 × 10^45
2. × 10^45
1.5 × 10^45
1. × 10^45
5. × 10^44
0.2
0.4
0.6
0.8
1.0
1.2
1.4
1.6
1.8
r/R




```
=== SECTION 8: PEAK POSITION OF P(r,0) ===
dP/dr=0  =>  rho^3 - rho^2 - 4*rho + 1 = 0  (rho=r/R)
All roots:
  rho = -1.699628  =>  r = -7.1988e+22 m  (outside [0,2R])
  rho = +0.239123  =>  r = 1.0128e+22 m  <-- physical
  rho = +2.460505  =>  r = 1.0422e+23 m  (outside [0,2R])
Peak: rho* = 0.239123,  r* = 1.0128e+22 m,  P(r*,0) = 3.803997e+45
m^2
```




```
=== SECTION 9: VIRTUAL PARTICLE SPECIES COMPARISON ===
Species                    V [m/s]         nu_1 [Hz]              T_1
[s]
-------------------------------------------------------------------
-------------
Photon    (V=c)            2.9979e+08      3.641727e-04
2.745950e+03
Electron (V=c/137)         2.1883e+06      2.658195e-06
3.761951e+05
Phonon    (V=346 m/s)      3.4600e+02      4.203033e-10
2.379234e+09
Gamma     (V=1e9*c)        2.9979e+17      3.641727e+05
2.745950e-06

Phonon T_1 = 2.3792e+09 s  =  7.5393e+01 years

=== SECTION 10: SUMMARY VERIFICATION TABLE ===
```

```
Quantity                                Formula                         Value

------------------------------------------------------------------------
-----------------
R (onduscular radius)                   given
4.235519e+22
sqrt(R)                                 sqrt(R)
2.058038e+11
k_1                                     pi/(2R)
3.708628e-23
omega_1                                 pi/(2*sqrt(R))
7.632496e-12
T_1 photon                              4*sqrt(R)/c
2.745950e+03
P(0,0)                                  2R^2
3.587923e+45
P(2R,0)                                 BC1=0
0.000000e+00
dP/dr|r=0                               BC2=R
4.235519e+22
rho* (flux peak)                        cubic root
0.239123
r* = rho**R                             physical peak
1.012811e+22
Norm integral I                         int_0^{2R} P dr
1.771651e+68
N_phys = 1/I                            normalization const
5.644453e-69

=== END OF MAPLE TRANSCRIPT (v3 — all errors fixed) ===
```